

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-2 6



Submitted By
PRINCE KUMAR
ERP no.-RU-25-11734

**Bachelor of Technology in Computer Science &
Engineering association with GOOGLE**

Under the Guidance of
Mr. ASHISH

***SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY***

BHILAI , C G

(April 2026)

Bank Account Management System

Design and Implementation using Object-Oriented Programming

OOP Project Report — Java

Subject	Object Oriented Programming with Java
Subject Code	RU-100-01-00012
Project Title	Bank Account Management System
Session	2025–26
Submission Date	April 2026
Technology	Core Java, OOP, Encapsulation, Abstraction

INDEX

Sr. No.	Title	Page No.
01	Abstract	3
02	Introduction	4
03	Scope & Limitations	6
04	System Requirements	7
05	Methodology & Approach	8
06	System Design & Architecture (Flowchart)	9
07	Implementation Details	11
08	Sample Input / Output & Test Cases	14
09	Project Timeline (Gantt Chart)	16
10	Conclusion	18
11	References	19

ABSTRACT

Banking systems are fundamental to financial management for individuals and organizations alike. This project focuses on building a Bank Account Management System using Object-Oriented Programming (OOP) concepts such as encapsulation, abstraction, and data hiding.

The system defines a BankAccount class with heavily encapsulated private balance attributes, providing public getter and setter methods to facilitate safe deposit and withdrawal operations while strictly validating all transaction amounts. The design ensures that no invalid or unauthorized access to the account balance is possible.

Key OOP Concepts Demonstrated: Encapsulation, Data Hiding, Abstraction, Getter/Setter Methods, Input Validation

Technology: Java (Core), Object-Oriented Design Patterns, Git / GitHub for Version Control

1. INTRODUCTION

1.1 Background & Context

Bank account management is one of the most critical operations in any financial institution. Traditional or poorly designed systems often fail to protect sensitive account data such as balances, leading to unauthorized access, data corruption, and financial fraud. As the demand for secure, scalable, and maintainable software grows, the need for properly designed object-oriented banking systems becomes essential.

The Bank Account Management System addresses this need by providing a secure, object-oriented solution capable of managing account balances through strictly controlled access, eliminating the risk of direct attribute manipulation.

1.2 Problem Statement

Current banking application designs face several challenges:

- Direct access to balance fields allows unauthorized modification without validation
- Lack of encapsulation exposes sensitive financial data to other classes and modules
- No centralized validation mechanism for deposit and withdrawal operations
- No structured mechanism to prevent negative balances or invalid transaction amounts

1.3 Proposed Solution

This project implements a Java-based Bank Account Management System that:

- Defines a BankAccount class with private balance attributes for complete data hiding
- Exposes controlled public getter and setter methods for safe read/write operations
- Validates all deposit and withdrawal amounts before modifying the balance
- Demonstrates encapsulation by preventing direct access to the balance field
- Is designed for easy extension with features like interest calculation and multiple account types

2. PROJECT OBJECTIVES

Primary Objectives

1. System Development

- Design and implement a functional Bank Account Management System in Java
- Create well-structured class hierarchies following OOP principles
- Implement type-specific deposit and withdrawal logic with strict validation

2. OOP Implementation

- Demonstrate encapsulation through private balance with public getter/setter methods
- Implement data validation inside setter methods for all transactions
- Use constructor initialization for setting account holder details and initial balance
- Override toString() for formatted account display

3. Functionality

- Enable deposit operations with positive amount validation
- Enable withdrawal operations with sufficient balance and positive amount checks
- Display comprehensive account details and transaction summary

Secondary Objectives

4. Code Quality

- Write clean, well-commented Java code following naming conventions
- Implement proper boundary checks and error handling
- Ensure code readability and maintainability

5. Version Control & Documentation

- Maintain code on GitHub repository with meaningful commits
- Write Javadoc comments for all classes and methods
- Produce a comprehensive project report and viva presentation

Expected Learning Outcomes

Competency	Expected Achievement
Encapsulation	Design and implement private attributes with public getter/setter in Java
Data Hiding	Prevent direct access to balance and enforce controlled modification
Input Validation	Validate all transaction amounts before modifying internal state
Constructor Design	Correctly initialize account holder info and opening balance
OOP Design	Build modular, extensible, real-world software systems

3. SYSTEM SCOPE & LIMITATIONS

3.1 Scope

In Scope:

- Account information management (account holder name, account number, balance)
- Deposit operation: validates amount > 0 before adding to balance
- Withdrawal operation: validates amount > 0 and sufficient balance before deducting
- Encapsulated private balance with public `getBalance()` and `setBalance()` methods
- Console-based account display and transaction summary
- Support for multiple account objects in a single run

Out of Scope:

- Database persistence (currently uses in-memory / hardcoded data)
- Graphical User Interface (GUI)
- Web-based or mobile deployment
- User authentication and role-based access control
- Interest calculation, tax deductions, and statement generation
- Multi-branch or multi-bank support

3.2 Limitations

Limitation	Current Status	Future Resolution
In-Memory Data	Hardcoded account objects; data lost on exit	MySQL / SQLite database integration
Account Types	Only BankAccount supported	Add SavingsAccount, CurrentAccount, LoanAccount
Console Only	CLI output only	Desktop / Web UI (JavaFX / Spring Boot)
Single User	No login or role management	Role-based access control (Admin/Customer)
No Interest Logic	Static balance only, no interest accrual	Interest rate slabs, compound interest
Fixed Attributes	Static attribute set per class	Dynamic attribute configuration

4. SYSTEM REQUIREMENTS

Hardware Requirements

Requirement	Specification
Processor	Intel Core i3 or equivalent (minimum)
RAM	2 GB minimum (4 GB recommended)
Storage	50 MB free disk space
Display	Standard monitor with terminal / console support

Software Requirements

Requirement	Version / Details
Operating System	Windows 10/11, Linux (Ubuntu 20+), macOS 12+
Java Development Kit (JDK)	JDK 8 or higher (JDK 17 LTS recommended)
Java Runtime Environment (JRE)	Java 8 or higher
IDE (Recommended)	IntelliJ IDEA, Eclipse, VS Code with Java Extension
Build Tool (Optional)	Maven 3.6+ or Gradle 7.0+
Version Control	Git 2.0+
Code Repository	GitHub (github.com)

5. METHODOLOGY & APPROACH

5.1 Development Methodology: Object-Oriented Design

The project follows a structured step-by-step OOP methodology across four phases:

Phase 1 — Analysis & Design

- Identify core entities: BankAccount (primary class)
- Define class attributes, getter/setter methods, and method contracts
- Design validation interface via deposit() and withdraw() methods

Phase 2 — Implementation

- Implement BankAccount class with private balance and accountHolder fields
- Create public getter methods: getBalance(), getAccountHolder(), getAccountNumber()
- Create public setter: setBalance() with positive amount validation
- Implement deposit() method with validation and balance update logic
- Implement withdraw() method with balance sufficiency check and validation

Phase 3 — Testing & Validation

- Unit test each method independently
- Test encapsulation by verifying private balance is not directly accessible
- Validate transaction output against expected values
- Boundary testing: zero amount, negative amount, over-withdrawal

Phase 4 — Documentation & Deployment

- Javadoc comments for all classes and methods
- README.md with setup and run instructions
- GitHub repository setup with commit history
- Project report and viva preparation

6. SYSTEM DESIGN & ARCHITECTURE

6.1 Class Architecture

The system is composed of one primary class with a clean, secure design:

Class	Type	Responsibilities
BankAccount	Primary Class (Encapsulated)	Private attributes: balance, accountHolder, accountNumber; provides deposit(), withdraw(), getBalance(), setBalance()
Main / BankingSystem	Entry Point	Creates BankAccount objects, performs transactions, displays results and account summary

6.2 Class Design Details

BankAccount Class:

- Attributes: private String accountHolder, private String accountNumber, private double balance
- Constructor: BankAccount(String accountHolder, String accountNumber, double initialBalance)
- Getters: getAccountHolder() : String, getAccountNumber() : String, getBalance() : double
- Setter: setBalance(double amount) with positive-amount validation
- Methods: deposit(double amount) : void, withdraw(double amount) : void, toString() : String

deposit() Method:

- Validates that the deposit amount is greater than zero
- Adds validated amount to the private balance field
- Prints confirmation message with updated balance

withdraw() Method:

- Validates that the withdrawal amount is greater than zero
- Checks that balance is sufficient to cover the withdrawal

- Deducts validated amount from the private balance field
- Prints error message if validation fails; confirmation if successful

6.3 System Flowchart

The following flowchart illustrates the complete execution flow of the Bank Account Management System, from system initialization through transaction type determination, amount validation, balance modification, and final account summary generation.

7. IMPLEMENTATION DETAILS

7.1 BankAccount Class — Core Entity

The BankAccount class encapsulates all sensitive account attributes and defines deposit() and withdraw() methods with strict validation. Using private attributes with public getters and setters enforces encapsulation and data integrity.

- Private attributes enforce data hiding and controlled access
- Constructor initializes all account properties including opening balance
- deposit() and withdraw() serve as the secure transaction contract methods
- toString() provides a formatted string representation for display

7.2 Deposit Method — Safe Credit Operation

The deposit() method validates and applies credit operations to the account balance.

Method	Purpose	Logic
Constructor	Initialize account	Set accountHolder, accountNumber, balance
deposit(double)	Credit account safely	Validate amount > 0; balance += amount
toString()	Display account info	Name Account No. Balance

7.3 Withdraw Method — Safe Debit Operation

The withdraw() method validates and applies debit operations with dual-check logic.

Method	Purpose	Logic
withdraw(double)	Debit account safely	Validate amount > 0 AND balance >= amount; balance -= amount
getBalance()	Read-only balance access	return balance (private field exposed safely)
setBalance(double)	Controlled balance set	Validate non-negative before assignment

7.4 Encapsulation in Main Method

In the main method, BankAccount objects are used exclusively through public methods, demonstrating proper encapsulation. The JVM enforces access control at compile time, preventing direct balance manipulation.

```
BankAccount acc1 = new BankAccount("Rahul", "ACC-1001", 10000.00);
```

```
BankAccount acc2 = new BankAccount("Priya", "ACC-1002", 5000.00);
```

```
acc1.deposit(5000.00);    // balance becomes 15000.00
```

```
acc1.withdraw(3000.00);  // balance becomes 12000.00
```

```
acc2.deposit(2000.00);   // balance becomes 7000.00
```

```
acc2.withdraw(9000.00);  // REJECTED: insufficient balance
```

8. SAMPLE INPUT / OUTPUT & TEST CASES

8.1 Sample Program Output

=== Bank Account Management System ===

Deposited: 5000.00 | New Balance: 15000.00

Withdrawn: 3000.00 | New Balance: 12000.00

Deposited: 2000.00 | New Balance: 7000.00

Error: Insufficient balance for withdrawal of 9000.00

ACCOUNT SUMMARY

Total Accounts : 2

Total Balance : 19000.00

8.2 Test Case Analysis

Test ID	Input / Scenario	Expected Output	Status	Result
TC-001	Create BankAccount (balance 10000.00)	Account initialized with balance 10000.00	✓ Pass	OK
TC-002	Deposit 5000.00 into acc1	Balance = 15000.00	✓ Pass	OK
TC-003	Withdraw 3000.00 from acc1	Balance = 12000.00	✓ Pass	OK
TC-004	Withdraw 9000.00 from acc2 (balance 7000.00)	Error: Insufficient balance	✓ Pass	OK
TC-005	Deposit negative amount (-500.00)	Error: Invalid deposit amount	✓ Pass	OK
TC-006	Withdraw zero amount	Error: Invalid withdrawal amount	✓ Pass	OK
TC-007	toString() on BankAccount object	Formatted account details displayed	✓ Pass	OK

9. PROJECT TIMELINE

9.1 Development Schedule

Phase	Task	Start	Duration
Analysis	Requirements gathering & class design	Week 1	2 days
Analysis	System design & architecture planning	Week 1	3 days
Development	BankAccount class implementation	Week 2	2 days
Development	deposit() & withdraw() method logic	Week 2	2 days
Development	Main application & transaction integration	Week 2–3	2 days
Testing	Unit testing & boundary validation	Week 3	3 days
Documentation	Javadoc & code comments	Week 3	2 days
Documentation	Project report creation	Week 3–4	3 days
Deployment	GitHub repository setup & commits	Week 4	1 day
Presentation	Viva preparation & mock Q&A	Week 4	2 days

9.2 Milestones

- Class Design Complete (Week 1)
- Core Implementation Done — BankAccount Class (Week 2)
- Validation & Testing Finished (Week 3)
- Documentation & Report Complete (Week 3–4)
- GitHub Submission & Project Delivery (Week 4)

10. FUTURE ENHANCEMENTS

10.1 Short-Term Improvements

Enhancement	Implementation	Timeline	Impact
More Account Types	Add SavingsAccount, CurrentAccount subclasses	Phase 2	Wider coverage
Interest Calculation	Add interest rate and compound interest logic	Phase 2	Realistic banking
Dynamic Input	Scanner / user input instead of hardcoded data	Phase 2	Interactive system
Transaction History	Log each transaction with timestamp	Phase 2	Better traceability
Account Statement	Print formatted transaction statement per account	Phase 2	Professional output

10.2 Medium-Term Extensions

- Database Integration (MySQL / SQLite) for persistent account storage
- File I/O for exporting transaction records to CSV / JSON
- User authentication system with Admin and Customer roles
- Multi-account and multi-branch management and reporting
- Fund transfer between accounts with validation
- PDF statement generation using iText / Apache PDFBox

10.3 Long-Term Vision

1. Web Application

- Spring Boot REST API backend
- Angular / React frontend for customer dashboard
- Cloud deployment on AWS / Azure

2. Mobile Application

- Native Android / iOS account viewer
- Push notifications for transaction alerts

3. AI/ML Integration

- Fraud detection using transaction pattern analysis
- Anomaly detection for unauthorized access attempts
- Predictive analytics for customer financial behavior

10.4 Technology Roadmap

v1.0 (Now)	v2.0	v3.0	v4.0
Java CLI	Java + DB	Spring Boot	Microservices
Basic OOP	File I/O	Web API	Docker / CI-CD
Encapsulation	Authentication	Full-stack UI	Mobile App
Validation	More Types	PDF Statements	AI Features

11. CONCLUSION

11.1 Project Achievement Summary

The Bank Account Management System project has successfully demonstrated all core OOP principles within a practical, real-world context:

Solid OOP Foundation:

- Correctly implemented encapsulation with private balance and public getter/setter
- Demonstrated strict input validation inside deposit() and withdraw() methods
- Used constructor for clean initialization of all account attributes
- Clear separation of concerns with a well-defined BankAccount class

Functional System:

- Accurate balance computation for both deposit and withdrawal transactions
- Extensible design ready for new account types without existing code changes
- Professional console output with account and transaction summary

Professional Development Practices:

- Clean, readable, well-documented Java code
- Version control and GitHub integration
- Comprehensive testing and validation across all scenarios

11.2 Learning Outcomes

Competency	Achievement
OOP Principles	Deep understanding of encapsulation, abstraction, and data hiding
Java Programming	Mastery of classes, objects, constructors, and method design
Software Design	Experience with secure class design and extensible architecture
Testing	Unit and integration testing with boundary value analysis
Version Control	Proficiency with Git and GitHub workflows
Documentation	Ability to write professional reports and technical specifications

11.3 Final Statement

The Bank Account Management System effectively demonstrates competency in object-oriented programming, software engineering principles, and professional development practices. The project meets all learning objectives and provides a solid foundation for building advanced database-driven banking systems, enterprise financial platforms, and cloud-based fintech solutions.

REFERENCES

Books & Textbooks

- Horstmann, C. S., & Cornell, G. (2019). Core Java: Fundamentals (11th ed.). Prentice Hall.
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Sierra, K., & Bates, B. (2005). Head First Java (2nd ed.). O'Reilly Media.

Online Resources & Documentation

- Oracle Java Documentation (2024). Java 17 API Documentation. <https://docs.oracle.com/en/java/javase/17/>
- GeeksforGeeks (2024). Encapsulation and Data Hiding in Java. <https://www.geeksforgeeks.org/encapsulation-in-java/>
- Baeldung (2024). Java OOP Concepts. <https://www.baeldung.com/java-oop>
- GitHub Guides (2024). Hello World Tutorial. <https://guides.github.com/activities/hello-world/>

Industry Standards & Style Guides

- Google Java Style Guide (2024). <https://google.github.io/styleguide/javaguide.html>
- Git Documentation (2024). <https://git-scm.com/doc>
- GitHub Documentation (2024). <https://docs.github.com/>